

Practical No: 7

1. Title

Demonstration of JSP Implicit Objects

2. Aim

To study and demonstrate the use of implicit objects in JavaServer Pages (JSP) for handling client requests, responses, and application data in a web application.

3. Learning Objectives

- To understand the concept of implicit objects in JSP
- To identify different types of JSP implicit objects
- To learn how to use objects like request, response, and session
- To implement implicit objects in a JSP program
- To analyze how JSP handles client-server communication

4. Software Requirements

- Operating System: Windows 10/11 or Linux
- Programming Language: Java
- Web Technology: JSP and Servlets
- Web Server: Apache Tomcat
- IDE/Editor: VS Code / Eclipse
- Browser: Chrome / Firefox

5. Theory Background

JavaServer Pages (JSP) is a server-side technology used to create dynamic web applications. It allows embedding Java code within HTML pages and runs on a web server like Apache Tomcat.

JSP provides **implicit objects**, which are automatically available without declaration. These objects simplify development by giving direct access to request data, response handling, session management, and application context.

There are **9 implicit objects** in JSP:

- **request** – Handles client request data
- **response** – Sends response to client
- **out** – Writes output to browser
- **session** – Stores user data across requests
- **application** – Shares data across application
- **config** – Provides servlet configuration
- **pageContext** – Access to all JSP scopes
- **page** – Represents current JSP instance
- **exception** – Used for error handling

6. Assignment Questions

7.1. Design a JSP page that accepts a user's name from an HTML form and displays a personalized welcome message using the entered data. If the user submits the form without entering a name, the application should display a suitable message such as "Name cannot be empty."

Input:

Index.html

```
<body>
<h2>Enter Your Name</h2>
    <form action="welcome.jsp" method="post">
        Name: <input type="text" name="username">
        <br><br>
        <input type="submit" value="Submit">
    </form>
</body>
```

Welcome.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<head>
<title>Welcome Page</title>
</head>
<body>
<h2>Welcome Page</h2>
<%
String name = request.getParameter("username");
if(name == null || name.trim().equals("")) {
%>
<p style="color:red;">Name cannot be empty.</p>
<%
} else {
%>
<p>Welcome, <%= name %>!</p>
<%
}
%>
</body>
</html>
```

Output:

Welcome Page Welcome Page

Welcome, Shriyash!

Name cannot be empty.

Enter Your Name

Name:

7.2. Create a login page where the user enters a username. Store the username in a session and display a welcome message on multiple pages without asking again. Develop additional pages such as a home page or profile page that retrieve the username from the session and display a personalized welcome message. The application should also include a logout mechanism that invalidates the session and redirects the user back to the login page.

Input:

Login.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<body>
    <h2>Login Page</h2>
    <form action="home.jsp" method="post">
        Username: <input type="text" name="username">
        <br><br>
        <input type="submit" value="Login">
    </form>
</body>
</html>
```

Home.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<body>
    <%
        String user = request.getParameter("username");
        if(user != null && !user.trim().equals("")) {
            session.setAttribute("user", user);
        }
        String username = (String) session.getAttribute("user");
        if(username == null) {
            response.sendRedirect("login.jsp");
        }
    %>
    <h2>Welcome, <%= username %>!</h2>
```

```
<a href="profile.jsp">Go to Profile</a><br>
<a href="logout.jsp">Logout</a>
</body>
</html>
```

Profile.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<!DOCTYPE html>
<html>
<body>
    <%
        String username = (String) session.getAttribute("user");
        if(username == null) {
            response.sendRedirect("login.jsp");
        }
    %>
    <h2>User Profile</h2>
    <p>Welcome, <%= username %>!</p>
    <a href="home.jsp">Back to Home</a><br>
    <a href="logout.jsp">Logout</a>
</body>
</html>
```

Logout.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%
    session.invalidate();
    response.sendRedirect("login.jsp");
%>
```

Output:

Login Page

Username:

Welcome, Shriyash_Patil!

[Go to Profile](#)
[Logout](#)

User Profile

Welcome, Shriyash_Patil!

[Back to Home](#)
[Logout](#)

7.3. Develop a JSP application that tracks and displays the total number of visitors accessing a webpage. The application should use the application implicit object to maintain a global counter that is shared among all users. Each time the page is accessed, the application should check whether the counter already exists; if not, it should initialize it. Otherwise, it should increment the existing value by one and display the updated visitor count on the page. The counter should persist across multiple users and sessions, demonstrating how application-level data is shared in JSP.

Input:

```
<%@ page language="java" %>
<html>
    <body>
        <h2>Visitor Counter</h2>
        <%
            Integer count = (Integer) application.getAttribute("visitorCount");
            if(count == null) {
                count = 1;
            }
            else { count =
                count + 1;
            }
            application.setAttribute("visitorCount", count);
        %>
        <p>Total Visitors: <%= count %></p>
    </body>
</html>
```

Output:

Visitor Counter

Total Visitors: 24

7.4. Create a JSP page that checks user input and redirects to different pages (success/error) based on the input.

Input:

Index.html

```
<%@ page language="java" %>
<html>
```

```
<body>
<h2>Login Form</h2>
<form action="check.jsp" method="post">
    Username: <input type="text" name="username"><br><br>
    Password: <input type="password" name="password"><br><br>
    <input type="submit" value="Submit">
</form>
</body>
</html>
```

Check.jsp

```
<%@ page language="java" contentType="text/html; charset=UTF-8"
    pageEncoding="UTF-8"%>
<%
String user = request.getParameter("username");
String pass = request.getParameter("password");

if(user != null && pass != null && user.equals("admin") && pass.equals("1234")) {
    response.sendRedirect("success.jsp");
} else {
    response.sendRedirect("error.jsp");
}
%>
```

Error.jsp

```
<%@ page language="java" %>
<html>
<body>
<h2 style="color:red;">Login Failed!</h2>
<p>Invalid Username or Password.</p>
</body>
</html>
```

Success.jsp

```
<%@ page language="java" %>
<html>
<body>

<h2 style="color:green;">Login Successful!</h2>
<p>Welcome, Admin!</p>
</body>
</html>
```

Output:

Login Form

Username:

Password:

Login Successful! **Login Failed!**

Welcome, Admin!

Invalid Username or Password.

7. Conclusion

The experiment on JavaServer Pages (JSP) implicit objects was successfully implemented. We learned how implicit objects like request, response, session, and application are used to handle user input, manage sessions, perform redirection, and share data across multiple users. Different JSP programs such as form validation, session tracking, visitor counter, and page redirection were developed and executed successfully. The results show that JSP implicit objects play an important role in simplifying web application development and improving efficiency. Hence, the objectives of the experiment were achieved successfully.